

Aidan Cusa
Professor Sable
ECE467 - Natural Language Processing
12/20/2024

Financial Sentiment Analyzer

For my final project, I made a financial sentiment analysis model. The objective is to take a sentence relating to finance and predict whether it has a positive, negative, or neutral tone. For example, if the model works correctly, the sentence below should have a negative sentiment attached to it.

“The company has suffered large losses” ⇒ **negative**

The dataset that I decided to use was the Financial Phrase Bank v.1.0 (2013) as it consisted of 4840 total sentences annotated by “16 people with adequate background knowledge on financial markets” (3 researchers, 13 master’s students from the Aalto University School of Business). I found it online when looking for labeled financial data [on hugging face](#). Each sentence had 5 to 8 annotations (positive, negative neutral) so naturally, there was going to be some overlap in opinion on what the label should be.

Thus the dataset was split into:

- Sentences with 100% Agreement
- Sentences with 75% Agreement
- Sentences with 66% Agreement
- Sentences with 50% Agreement

Sentences with 100% agreement means that every person who annotated the sentence agreed with what the label should be. That goes for 75%, 66%, and 50% agreement sentences as well. It’s also important to note that each % agreement dataset also includes the data in the higher % agreement as well. By this, I mean that the 66% dataset also includes data from the 75% agreement and 100% agreement datasets. So it really should be called sentences with $\geq 66\%$ agreement are included in the dataset.

To program my model, I referenced this [GitHub repository](#) which is a more general binary sentiment analyzer. I moreso used this as a reference to understand where to start and I fit it to my objective and dataset.

Experimentation

In total, I performed 11 different experiments, trying different pre-trained BERT models, datasets (50%, 66%, 75%, 100% agree), learning rates, optimizers, and loss functions. The results are shown below in Appendix I, displaying the results of every iteration that I tried. Here's a summary of what I tried and what's used in my final system with the **highlighted information in the final models:**

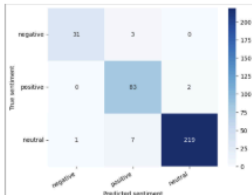
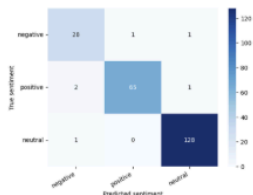
- I trained my original system using the BERT-base-cased model which performed relatively well on the test set (93.35% accuracy). However, when I was reading through slides of a presentation I found online regarding sentiment analysis (seen here web.stanford.edu/class/cs224u/2014/slides/cs224u-2014-lec15-sentiment.pdf) I realized that possibly fine-tuning an already fine-tuned system for sentiment analysis on my dataset could improve accuracy. So instead of using the base BERT model, I switched to the **distilbert-base-uncased-finetuned-sst-2-english** model which is a distilbert model already fine-tuned on binary sentiment analysis.
 - This distilbert model was fine-tuned on the Stanford Sentiment Treebank (SST-2) dataset, which is designed for binary classification (positive vs. negative)
 - I then fine-tuned this existing model for 3-outcome classification (positive, negative, and neutral)
 - Without changing any other parameters, this approach increased my accuracy across my validation and testing set by a significant margin and it took less time to train each epoch. **Thus, my further testing with different hyperparameters used the distilbert model as its base (and it's used in my final models).**
- I tried fine-tuning the model on each dataset to see if one particular dataset had the highest accuracy (fixing the parameters). Unsurprisingly, the 100% agreement dataset had the highest accuracy, with the 75%, 50%, and 66% agreement datasets following in that order.
 - To see the 100% and 75% agreement datasets having the highest accuracy is unsurprising as there should be less debate over the label in sentences that everyone agrees should have the same label. Thus, the testing accuracy should naturally be higher for the models that were trained and tested on these datasets.

- Note that I did try testing the 100% agreement model on the 75% agreement dataset (I forgot to save these results in the table by accident) though I remember seeing accuracy of ~97%. Although this is high accuracy, I think that this approach could be biased as the 75% agreement dataset also includes the data from the 100% agreement dataset. Thus, when testing, it's already seen most of the examples when training on the 100% training dataset. So I'm not trusting this approach or considering it.
 - It's also interesting to see the 66% agreement dataset perform worse on the 50% agreement. This could be due to the split of data for the test set of the 50% agreement dataset having more obvious examples of positive/negative sentences than the 66% agreement dataset's test set. This could also be supported by the low validation accuracy of the 50% agreement model compared to the 66% agreement model.
 - I actually have two final models, where I use both the 75% agreement and the 100% agreement datasets.
- For learning rates, I started with $2e-5$ (the standard value supposedly) but also tried $9e-4$, $1e-5$, and $3e-5$ as well. I varied this parameter on the 75% agreement dataset.
 - Testing accuracy for LR = $9e-4$: 62.05%
 - Testing accuracy for LR = $1e-5$: 96.53%
 - Testing accuracy for LR = $2e-5$: 96.24%
 - Testing accuracy for LR = $3e-5$: 93.93%
 - I think it's very strange to see the sharp-dip in testing accuracy from such a slight change in learning rate (from $9e-4$ to $1e-5$) though potentially it could have converged to a value closer to the 90%^s if there were a greater number of epochs.
 - Overall, I chose the learning rate of $2e-5$ for my final model as, although it had a slightly lower testing accuracy than $1e-5$, its validation accuracy converged to a value of more than 1% higher than that of $1e-5$. Thus, I think in a practical setting with more variation, the $2e-5$ learning rate model could perform better.
 - I tried two different optimizers, AdamW and RMSprop, two popular optimizers in deep learning that work well with sentiment analysis. Although RMSprop is a subset and

predecessor to Adam, I read on a [reddit forum](#) that it could sometimes work better in practice. Thus only varying the optimizer, AdamW had a better testing (and validation average accuracy) than RMSprop, so I decided to **use AdamW** in my final model.

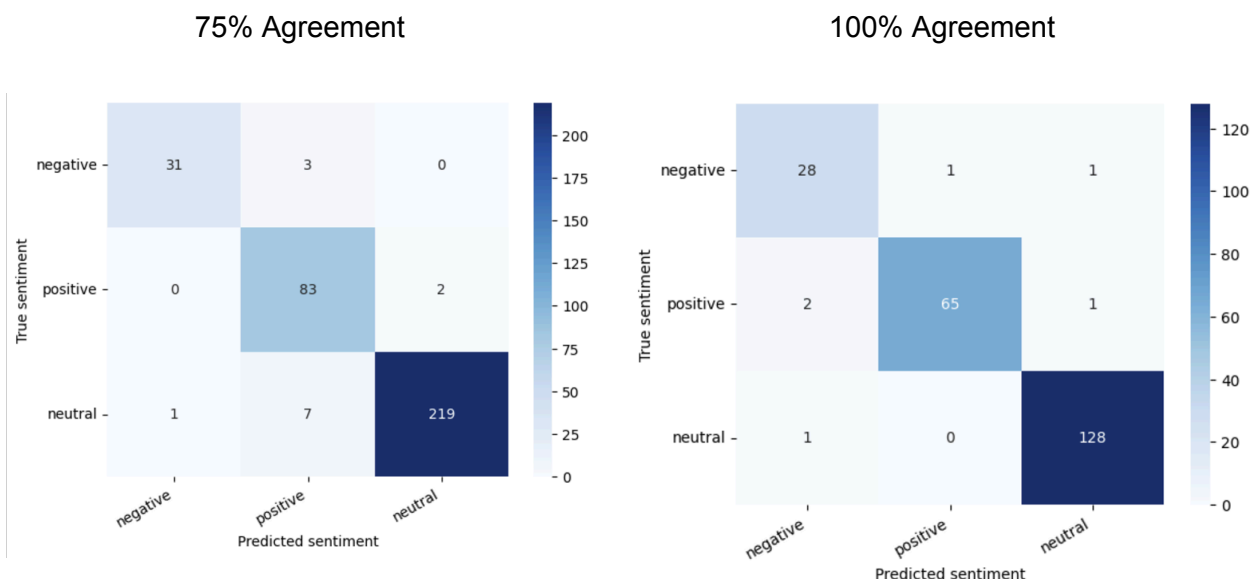
- I tried two different loss functions, Cross Entropy and Focal Loss. The general consensus is that Cross Entropy is more common to sentiment analysis though and that can be backed up by it having a slightly higher testing accuracy and average validation accuracy than Focal Loss, but not by much more. However, I tried it again with different learning rates (again using the 75% agreement dataset but LR = 1e5) and this time, the testing accuracy and average validation accuracy was much higher for Cross Entropy than for focal loss. Thus, I chose to **use Cross Entropy in my final model**.

Final results: Two models, one from iteration 2 and iteration 9.

	Iteration 2	Iteration 9
Train Accuracy	98.77%	99.89%
Validate Accuracy	94.49%	97.79%
Test Accuracy	96.24%	97.36%
Epochs	5	5
Tokenizer (off of <u>pretrained</u> model)	DistilBERT <u>tokenizer</u>	DistilBERT <u>tokenizer</u>
Pre-trained model	<u>distilbert-base-uncased-finetuned-sst-2-english</u>	<u>distilbert-base-uncased-finetuned-sst-2-english</u>
Dataset	75% Agree	100% Agree
Learning Rate	2e-5	2e-5
Optimizer	AdamW	AdamW
Loss Function	Cross Entropy	Cross Entropy
Avg. Time/Epoch	8:36	5:36
Confusion Matrix		

Both of these models use the distilbert model fine-tuned on binary classification (SST-2) as a base (and then I further fine-tune it from there), a learning rate of $2e-5$, the AdamW optimizer, and the cross entropy loss function. The only major difference is the dataset used to train the model.

- **Model 1** uses: 75% agreement dataset
- **Model 2** uses: 100% agreement dataset



The confusion matrices above show how well the models performed. I think what's most important is not predicting positive when the true sentiment is negative as that is clearly wrong (and vice versa). Both models seemed to do this three times which I'd say is a better look for the 75% agreement dataset since it has fewer examples and some of them are debatable.

If I had to choose one model, I would choose the 75% agreement model as I feel like it encompasses more examples of what could actually appear in the real world, especially today where more/updated financial lingo could exist (keep in mind dataset was made in 2013).

I feel like if there is something that shouldn't have a lot of new words though, like say a company's quarterly report where sentences are more clear, the 100% agreement model would be more useful as it's exposed to language that's more clear cut and there shouldn't be any debate as to what the sentiment of a sentence should be. Also in more professional documents

like those, I feel like there's less of a chance the model will encounter something it's never seen before that's relevant to the company/security's performance.

Thus, overall if we're just feeding single sentences into the model and trying to predict the sentiment, I'd choose the 75% agreement model. However, if we're analyzing something more professional that has less debate and we're trying to get overall sentiment (a topic I didn't get to explore in this project but is one I'd like to explore) then the 100% agreement model would be more useful.

Play with the model!

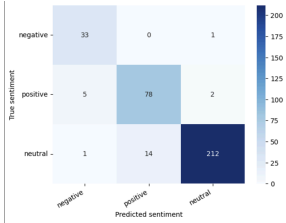
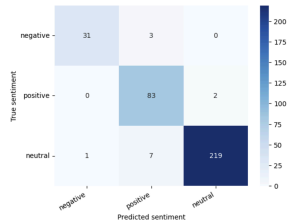
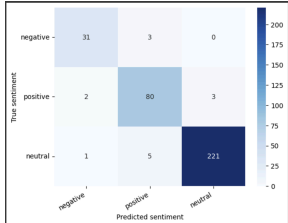
I saved the states of the 75% agreement and the 100% agreement models (iterations 2 and 9) that performed the best overall. I will include these .bin files in the email that I sent to you.

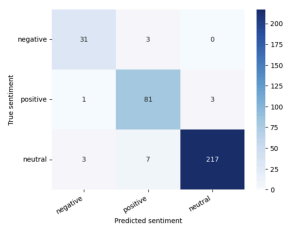
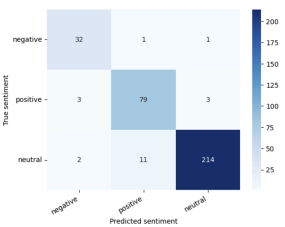
To load them in and try the model, please:

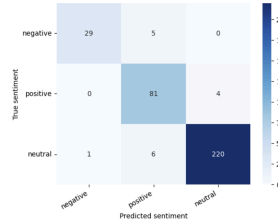
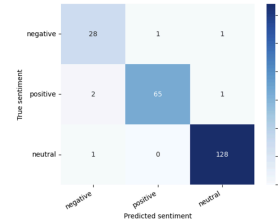
- In the Google Colab, go to the second to last cell and run it. This sets up all the necessary variables that we need to use to load and play with the model.
 - This information is all previously defined, but if you don't want to run every cell you can just run this cell.
- Next in the last cell (directly after the one just run), enter the model that you want to use in the line: `model.load_state_dict(torch.load('model_name'))`
 - For the 100% agreement model use: `best_model_state_100.bin`
 - For the 75% agreement model: `best_model_state_75.bin`
- Then enter in the sentence you want to predict the sentiment for in the `review_text` variable and run the cell.
- See the output for the predicted sentiment!

```
Truncation was not explicitly activated but max_l  
Review text: The company has suffered large losses  
Sentiment : negative
```

Appendix I. Results of the different iterations tried with different parameters.

	Iteration 1	Iteration 2	Iteration 3																																																
Train Accuracy	95.15%	98.77%	99.86%																																																
Validate Accuracy	92.46%	94.49%	94.20%																																																
Test Accuracy	93.35%	96.24%	95.95%																																																
Epochs	2	5	5																																																
Tokenizer (off of pretrained model)	BERT tokenizer	DistilBERT tokenizer	DistilBERT tokenizer																																																
Pre-trained model	bert-base-cased	distilbert-base-uncased-finetuned-sst-2-english	distilbert-base-uncased-finetuned-sst-2-english																																																
Dataset	75% Agree	75% Agree	75% Agree																																																
Learning Rate	2e-5	2e-5	2e-5																																																
Optimizer	AdamW	AdamW	AdamW																																																
Loss Function	Cross Entropy	Cross Entropy	Focal Loss																																																
Avg. Time/Epoch	17:23	8:36	8:34																																																
Confusion Matrix	 <table border="1"> <thead> <tr> <th></th><th>negative</th><th>positive</th><th>neutral</th></tr> </thead> <tbody> <tr> <th>negative</th><td>33</td><td>0</td><td>1</td></tr> <tr> <th>positive</th><td>5</td><td>78</td><td>2</td></tr> <tr> <th>neutral</th><td>1</td><td>14</td><td>212</td></tr> </tbody> </table>		negative	positive	neutral	negative	33	0	1	positive	5	78	2	neutral	1	14	212	 <table border="1"> <thead> <tr> <th></th><th>negative</th><th>positive</th><th>neutral</th></tr> </thead> <tbody> <tr> <th>negative</th><td>31</td><td>3</td><td>0</td></tr> <tr> <th>positive</th><td>0</td><td>83</td><td>2</td></tr> <tr> <th>neutral</th><td>1</td><td>7</td><td>219</td></tr> </tbody> </table>		negative	positive	neutral	negative	31	3	0	positive	0	83	2	neutral	1	7	219	 <table border="1"> <thead> <tr> <th></th><th>negative</th><th>positive</th><th>neutral</th></tr> </thead> <tbody> <tr> <th>negative</th><td>31</td><td>3</td><td>0</td></tr> <tr> <th>positive</th><td>2</td><td>80</td><td>3</td></tr> <tr> <th>neutral</th><td>1</td><td>5</td><td>221</td></tr> </tbody> </table>		negative	positive	neutral	negative	31	3	0	positive	2	80	3	neutral	1	5	221
	negative	positive	neutral																																																
negative	33	0	1																																																
positive	5	78	2																																																
neutral	1	14	212																																																
	negative	positive	neutral																																																
negative	31	3	0																																																
positive	0	83	2																																																
neutral	1	7	219																																																
	negative	positive	neutral																																																
negative	31	3	0																																																
positive	2	80	3																																																
neutral	1	5	221																																																

	Iteration 4	Iteration 5	Iteration 6																																
Train Accuracy	98.66%	99.75%	98.73%																																
Validate Accuracy	93.62%	92.75%	93.62%																																
Test Accuracy	95.09%	93.93%	96.53%																																
Epochs	5	5	5																																
Tokenizer (off of pretrained model)	DistilBERT tokenizer	DistilBERT tokenizer	DistilBERT tokenizer																																
Pre-trained model	distilbert-base-uncased-finetuned-sst-2-english	distilbert-base-uncased-finetuned-sst-2-english	distilbert-base-uncased-finetuned-sst-2-english																																
Dataset	75% Agree	75% Agree	75% Agree																																
Learning Rate	2e-5	3e-5	1e-5																																
Optimizer	RMSprop	AdamW	AdamW																																
Loss Function	Cross Entropy	Cross Entropy	Cross Entropy																																
Avg. Time/Epoch	8:31	8:31	8:34																																
Confusion Matrix	 Confusion Matrix for Iteration 4: <table border="1"> <thead> <tr> <th></th><th>Predicted negative</th><th>Predicted positive</th><th>Predicted neutral</th></tr> </thead> <tbody> <tr> <th>True negative</th><td>31</td><td>3</td><td>0</td></tr> <tr> <th>True positive</th><td>1</td><td>81</td><td>3</td></tr> <tr> <th>True neutral</th><td>3</td><td>7</td><td>217</td></tr> </tbody> </table>		Predicted negative	Predicted positive	Predicted neutral	True negative	31	3	0	True positive	1	81	3	True neutral	3	7	217	 Confusion Matrix for Iteration 5: <table border="1"> <thead> <tr> <th></th><th>Predicted negative</th><th>Predicted positive</th><th>Predicted neutral</th></tr> </thead> <tbody> <tr> <th>True negative</th><td>32</td><td>1</td><td>1</td></tr> <tr> <th>True positive</th><td>3</td><td>79</td><td>3</td></tr> <tr> <th>True neutral</th><td>2</td><td>11</td><td>214</td></tr> </tbody> </table>		Predicted negative	Predicted positive	Predicted neutral	True negative	32	1	1	True positive	3	79	3	True neutral	2	11	214	
	Predicted negative	Predicted positive	Predicted neutral																																
True negative	31	3	0																																
True positive	1	81	3																																
True neutral	3	7	217																																
	Predicted negative	Predicted positive	Predicted neutral																																
True negative	32	1	1																																
True positive	3	79	3																																
True neutral	2	11	214																																

	Iteration 7	Iteration 8	Iteration 9																																
Train Accuracy	99.20%	61.80%	99.89%																																
Validate Accuracy	92.46%	61.45%	97.79%																																
Test Accuracy	95.38%	62.05%	97.36%																																
Epochs	5	3	5																																
Tokenizer (off of pretrained model)	DistilBERT tokenizer	DistilBERT tokenizer	DistilBERT tokenizer																																
Pre-trained model	distilbert-base-uncased-finetuned-sst-2-english	distilbert-base-uncased-finetuned-sst-2-english	distilbert-base-uncased-finetuned-sst-2-english																																
Dataset	75% Agree	75% Agree	100% Agree																																
Learning Rate	1e-5	9e-4	2e-5																																
Optimizer	AdamW	AdamW	AdamW																																
Loss Function	Focal Loss	Cross Entropy	Cross Entropy																																
Avg. Time/Epoch	9:23	12:44	5:36																																
Confusion Matrix	 <table border="1"> <thead> <tr> <th></th><th>negative</th><th>positive</th><th>neutral</th></tr> </thead> <tbody> <tr> <th>negative</th><td>29</td><td>5</td><td>0</td></tr> <tr> <th>positive</th><td>0</td><td>81</td><td>4</td></tr> <tr> <th>neutral</th><td>1</td><td>6</td><td>229</td></tr> </tbody> </table>		negative	positive	neutral	negative	29	5	0	positive	0	81	4	neutral	1	6	229		 <table border="1"> <thead> <tr> <th></th><th>negative</th><th>positive</th><th>neutral</th></tr> </thead> <tbody> <tr> <th>negative</th><td>28</td><td>1</td><td>1</td></tr> <tr> <th>positive</th><td>2</td><td>65</td><td>1</td></tr> <tr> <th>neutral</th><td>1</td><td>0</td><td>129</td></tr> </tbody> </table>		negative	positive	neutral	negative	28	1	1	positive	2	65	1	neutral	1	0	129
	negative	positive	neutral																																
negative	29	5	0																																
positive	0	81	4																																
neutral	1	6	229																																
	negative	positive	neutral																																
negative	28	1	1																																
positive	2	65	1																																
neutral	1	0	129																																

	Iteration 10	Iteration 11	
Train Accuracy	94.13%	96.05%	
Validate Accuracy	90.02%	85.54%	
Test Accuracy	89.36%	94.80%	
Epochs	5	5	
Tokenizer (off of pretrained model)	DistilBERT tokenizer	DistilBERT tokenizer	
Pre-trained model	distilbert-base-uncased-finetuned-sst-2-english	distilbert-base-uncased-finetuned-sst-2-english	h
Dataset	66% Agree	50% Agree	e
Learning Rate	2e-5	2e-5	l
Optimizer	AdamW	AdamW	l
Loss Function	Cross Entropy	Cross Entropy	o
Avg. Time/Epoch	10:25	12:18	!
Confusion Matrix	